

LAPORAN PRAKTIKUM MATA KULIAH

METODE NUMERIK

PRAKTIKUM 04 – GALAT



DISUSUN OLEH:

L0325021 – DUTA LATHIIF HAQIM

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI DAN SAINS DATA

UNIVERSITAS SEBELAS MARET

2026

BAB I

ANALISIS SOURCE CODE

1. Analisis Source Code “GALAT DERET TAYLOR”

i. Analisis source code

```
# Menghitung nilai e^x
x = input('Input nilai x : ');
n = input('Input nilai n : ');

# Perhitungan nilai e^x dengan cara eksak
eksak = exp(x);

# Perhitungan nilai e^x dengan deret Taylor
p = 0;
for i = 0:n
    p = p + (x^i / factorial(i));
end

galat = abs(eksak - p);

fprintf('Eksak\t\t= %5.15f\n', eksak)
fprintf('Pendekatan\t= %5.15f\n', p)
fprintf('Error\t\t= %5.15f\n', galat)
```

Program ini menghitung nilai e^x dengan dua cara, lalu membandingkan keduanya untuk mengukur galat.

1. **Input:** input() meminta nilai x (titik yang dihitung) dan n (orde atau suku terakhir deret Taylor yang dipakai).
2. **Nilai eksak:** exp(x) memakai fungsi bawaan Octave sebagai nilai acuan (dianggap benar).
3. **Pendekatan deret Taylor (Maclaurin):** variabel p diawali 0, lalu loop for i = 0:n menjumlahkan suku demi suku: $e^x \approx \sum (x^i / i!)$ untuk i = 0 sampai n = 1 + x + x²/2! + x³/3! + ... + xⁿ/n! Deret ini dipotong setelah suku ke-n, sehingga hasilnya hanya pendekatan.
4. **Galat:** galat = abs(eksak - p) adalah galat absolut, yaitu selisih mutlak nilai eksak dan pendekatan. Karena deret dipotong, galat ini adalah **galat pemotongan (truncation error)**.
5. **Output:** fprintf dengan format %5.15f menampilkan 15 angka di belakang koma, dan \t memberi tab agar tampilannya rata.

Baris galat_Deret_Taylor at line 2 column 1 di bagian atas output kemungkinan besar pesan error dari percobaan sebelumnya (karakternya rusak saat di-copy). Enam percobaan yang Anda tampilkan berjalan normal, jadi bagian itu bisa dibuang dari laporan.

ii. Analisis hasil

```
Input nilai x : 0.3
Input nilai n : 0
Eksak          = 1.349858807576003
Pendekatan     = 1.000000000000000
Error          = 0.349858807576003
octave:11> galat_Deret_Taylor

Input nilai x : 0.3
Input nilai n : 1
Eksak          = 1.349858807576003
Pendekatan     = 1.300000000000000
Error          = 0.049858807576003
octave:12> galat_Deret_Taylor

Input nilai x : 0.3
Input nilai n : 2
Eksak          = 1.349858807576003
Pendekatan     = 1.345000000000000
Error          = 0.004858807576003
octave:13> galat_Deret_Taylor

Input nilai x : 0.3
Input nilai n : 3
Eksak          = 1.349858807576003
Pendekatan     = 1.349500000000000
Error          = 0.000358807576003
octave:14> galat_Deret_Taylor

Input nilai x : 0.3
Input nilai n : 4
Eksak          = 1.349858807576003
Pendekatan     = 1.349837500000000
Error          = 0.000021307576003
octave:15> |
```

Untuk $x = 0,3$, nilai eksaknya $e^{0,3} = 1,349858807576003$.

n	Pendekatan	Galat
0	1.000000000000000	0.349858807576003
1	1.300000000000000	0.049858807576003
2	1.345000000000000	0.004858807576003
3	1.349500000000000	0.000358807576003
4	1.349837500000000	0.000021307576003

Pengamatan:

1. **Galat menurun seiring bertambahnya n.** Setiap suku tambahan membuat hasil lebih dekat ke nilai eksak, dan galat turun dari orde 10^{-1} ($n = 0$) ke orde 10^{-5} ($n = 4$). Ini menunjukkan deret Taylor **konvergen** ke e^x .
2. **Pendekatan selalu lebih kecil dari nilai eksak.** Karena $x = 0,3 > 0$, semua suku deret positif. Suku yang dibuang selalu bernilai positif, jadi hasil pemotongan berada di bawah nilai sebenarnya.
3. **Galat kira-kira sama dengan suku pertama yang dibuang.** Sisa deret adalah $R_n = e^\xi \cdot x^{(n+1)} / (n+1)!$, dengan ξ di antara 0 dan x . Contohnya pada $n = 4$, suku berikutnya $x^5/5! = 0,3^5/120 = 0,00002025$, sangat dekat dengan galat hasil 0,0000213. Selisih kecilnya berasal dari suku-suku setelahnya, yaitu $x^6/6!$ dan seterusnya.
4. **Konvergensinya cepat.** Setiap penambahan satu suku memperkecil galat sekitar 7 sampai 17 kali. Rasionalnya membesar karena pembagiannya $(n+1)!$ tumbuh cepat, sedangkan $x = 0,3$ kecil sehingga pangkat x^n mengecil dengan cepat.
5. **Hasil pada $n = 0$ dan $n = 1$ sesuai hitungan manual.** Pada $n = 0$, $p = 1$. Pada $n = 1$, $p = 1 + 0,3 = 1,3$.

2. Analisis Source Code “A PERHITUNGAN SECARA EKSAK”

i. Analisis source code

```
clc;
clear;

n = 20;
jumlah_eksak = 0;

for i = 1:n
    jumlah_eksak = jumlah_eksak + 1/i;
end

fprintf('Hasil eksak = %.10f\n', jumlah_eksak);
```

Program ini menghitung **jumlah parsial deret harmonik** sampai suku ke-20:

$$H_{20} = 1 + 1/2 + 1/3 + \dots + 1/20 = \sum (1/i) \text{ untuk } i = 1 \text{ sampai } 20$$

1. `clc; clear;` membersihkan layar Command Window dan menghapus semua variabel di workspace, supaya hasilnya tidak terpengaruh program sebelumnya.
2. `n = 20;` adalah banyaknya suku yang dijumlahkan.
3. `jumlah_eksak = 0;` adalah nilai awal penjumlah (akumulator).
4. Loop `for i = 1:n` menambahkan $1/i$ ke akumulator pada setiap langkah. Indeks dimulai dari 1, bukan 0, supaya tidak terjadi pembagian dengan nol.
5. `fprintf('Hasil eksak = %.10f\n', ...)` mencetak hasil dengan 10 angka di belakang koma.

Catatan: nama variabel `jumlah_eksak` berarti hasil ini dipakai sebagai nilai acuan. Secara komputasi, Octave menghitungnya dengan bilangan floating-point presisi ganda (double), sehingga sebenarnya sudah mengandung **galat pembulatan** yang sangat kecil, sekitar 10^{-15} . Untuk tugas ini galat sekecil itu aman dianggap sebagai nilai eksak.

ii. Analisis hasil

```
Hasil eksak = 3.5977396571
octave:18> |
```

Output: $H_{20} = 3,5977396571$

Nilai ini benar. Nilai eksak secara rasional adalah $H_{20} = 55835135/15519504 \approx 3,59773965714368$, dan setelah dibulatkan ke 10 desimal hasilnya sama dengan output program.

Pengamatan:

1. **Deret harmonik divergen, tetapi jumlah parsialnya berhingga.** Untuk $n = 20$ nilainya hanya sekitar 3,6. Pertumbuhannya sangat lambat, mengikuti $H_n \approx \ln n + \gamma$, dengan $\gamma \approx 0,5772$ (konstanta Euler-Mascheroni).
2. **Verifikasi dengan pendekatan asimtotik:** $H_n \approx \ln n + \gamma + 1/(2n) - 1/(12n^2)$. Untuk $n = 20$ diperoleh $2,995732 + 0,577216 + 0,025 - 0,000208 \approx 3,597740$, sesuai dengan output program.

3. **Angka yang tampil dibatasi format %.10f.** Presisi double sebenarnya sekitar 15 sampai 16 digit signifikan, sehingga sepuluh angka desimal yang tercetak masih aman dari pengaruh galat pembulatan.
4. **Urutan penjumlahan mempengaruhi galat pembulatan.** Program menjumlahkan dari suku terbesar (1) ke suku terkecil (1/20). Penjumlahan dari suku terkecil ke terbesar biasanya sedikit lebih teliti, karena bilangan kecil tidak "hilang" saat ditambahkan ke akumulator yang sudah besar. Untuk $n = 20$ perbedaannya tidak terlihat, tetapi pada n yang sangat besar atau presisi tunggal (single) efeknya jelas.

Kesimpulan: jumlah 20 suku pertama deret harmonik adalah 3,5977396571, dan hasil ini konsisten dengan nilai rasional eksaknya maupun pendekatan asimtotiknya.

Biasanya kode seperti ini dilanjutkan dengan perbandingan terhadap penjumlahan presisi tunggal (single) atau penjumlahan terbalik, lalu galatnya dihitung. Kalau kode berikutnya seperti itu, silakan kirim.

3. Analisis Source Code “B DIBULATKAN”

i. Analisis source code

```
clc;
clear;

n = 20;
jumlah_bulat = 0;

for i = 1:n
    hasil_bagi = round((1/i) * 10000) / 10000;
    jumlah_bulat = jumlah_bulat + hasil_bagi;
end

fprintf('Hasil dibulatkan = %.10f\n', jumlah_bulat);
```

Program ini menghitung jumlah parsial deret harmonik yang sama seperti program sebelumnya ($n = 20$), tetapi **setiap suku dibulatkan dulu ke 4 angka desimal** sebelum dijumlahkan.

1. `clc; clear;` membersihkan layar dan workspace.
2. `n = 20;` adalah banyaknya suku, dan `jumlah_bulat = 0;` adalah akumulator.
3. Di dalam loop `for i = 1:n`, baris `hasil_bagi = round((1/i) * 10000) / 10000;` melakukan pembulatan ke 4 desimal:
 - o $(1/i) * 10000$ menggeser koma 4 tempat ke kanan.
 - o `round(...)` membulatkan ke bilangan bulat terdekat (0,5 ke atas dibulatkan ke atas).
 - o `/ 10000` mengembalikan koma ke posisi semula.

Contoh: $1/3 = 0,333333...$ menjadi $3333,33 \rightarrow 3333 \rightarrow \mathbf{0,3333}$.

4. `jumlah_bulat = jumlah_bulat + hasil_bagi`; menjumlahkan suku yang sudah dibulatkan.
5. `fprintf(... %.10f\n', ...)` mencetak hasil dengan 10 angka desimal.

Tujuan program ini adalah mensimulasikan **galat pembulatan (round-off error)**, seolah-olah perhitungan dilakukan dengan mesin yang hanya menyimpan 4 angka desimal. Hasilnya dibandingkan dengan `jumlah_eksak` dari program sebelumnya.

Karakter aneh di awal output (`φdz dz dz ...`) adalah sisa karakter rusak dari terminal saat di-copy, bukan hasil program. Sebaiknya dibuang dari laporan.

ii. Analisis hasil

```
Hasil dibulatkan = 3.5977000000
octave:19> |
```

Output: **3,5977000000**, sedangkan nilai acuan sebelumnya adalah 3,5977396571.

Galat:

- Galat absolut = $|3,5977396571 - 3,5977000000| = 0,0000396571 \approx 3,97 \times 10^{-5}$
- Galat relatif = $3,97 \times 10^{-5} / 3,5977 \approx 1,10 \times 10^{-5}$ (sekitar 0,0011%)

Pengamatan:

1. **Hasil ini bisa diverifikasi manual.** Suku-suku yang dibulatkan adalah 1,0000; 0,5000; 0,3333; 0,2500; 0,2000; 0,1667; 0,1429; 0,1250; 0,1111; 0,1000; 0,0909; 0,0833; 0,0769; 0,0714; 0,0667; 0,0625; 0,0588; 0,0556; 0,0526; 0,0500. Jumlahnya tepat **3,5977**.
2. **Angka nol di belakang (3.5977000000) bukan tanda hasil sangat teliti.** Karena tiap suku adalah kelipatan 0,0001, jumlahnya juga kelipatan 0,0001, sehingga desimal ke-5 sampai ke-10 pasti nol. Angka bermakna hanya 4 desimal.
3. **Suku yang tidak perlu dibulatkan** adalah 1, 1/2, 1/4, 1/5, 1/8, 1/10, 1/16, dan 1/20 (galat nol). Suku lain seperti 1/3, 1/7, atau 1/18 menyumbang galat pembulatan.
4. **Galat tiap suku maksimal $0,5 \times 10^{-4} = 5 \times 10^{-5}$.** Contoh: $1/7 = 0,142857...$ menjadi 0,1429 (galat $+4,3 \times 10^{-5}$), $1/3$ menjadi 0,3333 (galat $-3,3 \times 10^{-5}$), dan $1/18$ menjadi 0,0556 (galat $+4,4 \times 10^{-5}$).
5. **Galat-galat itu saling meniadakan sebagian.** Batas terburuk untuk 20 suku adalah $20 \times 5 \times 10^{-5} = 10^{-3}$, tetapi galat sebenarnya hanya $3,97 \times 10^{-5}$. Penyebabnya, sebagian suku dibulatkan ke atas (galat positif) dan sebagian ke bawah (galat negatif). Jumlah galat semua sukunya bernilai $-3,97 \times 10^{-5}$, sesuai dengan selisih yang diamati.
6. **Hasil pembulatan lebih kecil dari nilai eksak**, karena galat negatif sedikit lebih dominan daripada yang positif.

4. Analisis Source Code “B DIBULATKAN”

i. Analisis source code

```
clc;
clear;

n = 20;
i_vektor = 1:n;
jumlah_sum = sum(1 ./ i_vektor);

fprintf('Hasil tanpa loop (sum) = %.10f\n', jumlah_sum);
```

Program ini menghitung jumlah parsial deret harmonik yang sama ($n = 20$), tetapi dengan **vektorisasi**, tanpa loop for.

$H_{20} = \sum (1/i)$ untuk $i = 1$ sampai 20

1. `clc; clear;` membersihkan layar dan workspace.
2. `n = 20;` adalah banyaknya suku.
3. `i_vektor = 1:n;` membuat vektor baris [1, 2, 3, ..., 20] berukuran 1×20 .
4. `1 ./ i_vektor` adalah **pembagian elemen demi elemen** (element-wise). Hasilnya vektor [1/1, 1/2, ..., 1/20]. Tanda titik pada `./` wajib. Tanpa titik, `1 / i_vektor` dibaca sebagai pembagian matriks dan Octave akan menampilkan error ukuran (nonconformant arguments).
5. `sum(...)` menjumlahkan semua elemen vektor tersebut menjadi satu nilai skalar, yang disimpan di `jumlah_sum`.
6. `fprintf('... %.10f\n', ...)` mencetak hasil dengan 10 angka desimal.

Karakter aneh di awal output (𐀀𐀁𐀂 | ...) adalah sisa karakter rusak dari terminal, bukan hasil program. Sebaiknya dibuang dari laporan.

ii. Analisis hasil

```
Hasil tanpa loop (sum) = 3.5977396571
octave:20> |
```

Output: 3,5977396571

1. **Hasil sama dengan program loop.** Kedua program menghitung ekspresi matematika yang sama, jadi hasil pada 10 desimal identik. Nilai ini juga cocok dengan nilai rasional eksak $H_{20} = 55835135/15519504 \approx 3,59773965714$.
2. **Perbedaan di digit terakhir mungkin ada, tetapi tidak terlihat.** Pada presisi double, urutan penjumlahan yang berbeda bisa menghasilkan selisih sekitar 10^{-16} . Angka sekecil itu jauh di bawah 10 desimal yang dicetak. Kalau ingin membuktikannya, hitung `abs(jumlah_sum - jumlah_eksak)` atau tampilkan dengan `%.16f`.
3. **Galat terhadap nilai eksak tidak terlihat (≈ 0 pada 10 desimal).** Ini kontras dengan program pembulatan 4 desimal, yang galatnya $3,97 \times 10^{-5}$. Jadi galat pada program

sebelumnya berasal dari **pembulatan tiap suku yang disengaja**, bukan dari cara menjumlahkannya.

4. **Keunggulan vektorisasi**: kodenya lebih ringkas (3 baris inti, tanpa loop) dan jauh lebih cepat untuk n besar, karena Octave menjalankan operasi vektor pada level internal yang sudah dioptimalkan. Untuk $n = 20$ perbedaan waktunya tidak terasa, tetapi untuk $n = 10^6$ atau lebih, selisihnya jelas.
5. **Vektorisasi tidak mengubah ketelitian secara berarti**. Yang berubah hanya efisiensi dan gaya penulisan. Ketelitian tetap dibatasi presisi double (sekitar 15 sampai 16 digit signifikan).

Checking error ratios and convergence at each truncation.

i. Analisis source code

```
1 # Menghitung nilai sin(x)
2 x = input('Input nilai x : ');
3 n = input('Input nilai n : ');
4
5 # Perhitungan nilai sin(x) dengan cara eksak
6 eksak = sin(x);
7
8 # Perhitungan nilai sin(x) dengan deret Taylor
9 p = 0;
10 for i = 0:n
11     p = p + ((-1)^i * x^(2*i+1)) / factorial(2*i+1);
12 end
13
14 galat = abs(eksak - p);
15
16 fprintf('Eksak\t\t= %5.15f\n', eksak)
17 fprintf('Pendekatan\t= %5.15f\n', p)
18 fprintf('Error\t\t= %5.15f\n', galat)
```

Program ini menghitung $\sin(x)$ dengan **deret Maclaurin** dan membandingkannya dengan fungsi bawaan $\sin(x)$.

$\sin(x) \approx \sum [(-1)^i \cdot x^{(2i+1)} / (2i+1)!]$ untuk $i = 0$ sampai $n = x - x^3/3! + x^5/5! - x^7/7! + \dots$

1. `input()` meminta nilai x (dalam **radian**, bukan derajat) dan n (indeks suku terakhir).
2. `eksak = sin(x)` adalah nilai acuan dari fungsi bawaan Octave.
3. Loop `for i = 0:n` menjumlahkan suku demi suku:
 - $(-1)^i$ mengatur tanda bergantian (+, -, +, -, ...).
 - $x^{(2i+1)}$ adalah pangkat ganjil, karena $\sin(x)$ adalah fungsi ganjil.
 - `factorial(2*i+1)` adalah penyebut faktorial ganjil (1!, 3!, 5!, ...).
4. `galat = abs(eksak - p)` adalah galat absolut. Karena deretnya dipotong, galat ini adalah **galat pemotongan (truncation error)**.
5. `fprintf` mencetak hasil dengan 15 angka di belakang koma.

Catatan: nilai n di sini menyatakan indeks i terakhir, sehingga jumlah suku yang dipakai adalah $n + 1$. Untuk $n = 1$, suku yang dipakai adalah x dan $-x^3/3!$ (2 suku), dan polinom yang terbentuk berderajat $2n + 1 = 3$.

ii. Analisis hasil

```
octave:2> galat_sin

Input nilai x : 1
Input nilai n : 1
Eksak          = 0.841470984807897
Pendekatan     = 0.833333333333333
Error          = 0.008137651474563
octave:3> galat_sin

Input nilai x : 1
Input nilai n : 2
Eksak          = 0.841470984807897
Pendekatan     = 0.841666666666667
Error          = 0.000195681858770
octave:4> galat_sin

Input nilai x : 1
Input nilai n : 3
Eksak          = 0.841470984807897
Pendekatan     = 0.841468253968254
Error          = 0.000002730839643
octave:5> galat_sin

Input nilai x : 1
Input nilai n : 4
Eksak          = 0.841470984807897
Pendekatan     = 0.841471009700176
Error          = 0.000000024892280
octave:6> galat_sin

Input nilai x : 1
Input nilai n : 5
Eksak          = 0.841470984807897
Pendekatan     = 0.841470984648068
Error          = 0.000000000159828
octave:7> |
```

Untuk $x = 1$, nilai eksaknya $\sin(1) = 0,841470984807897$.

n	Pendekatan	Galat	Posisi terhadap eksak
1	0.833333333333333	0.008137651474563	lebih kecil
2	0.841666666666667	0.000195681858770	lebih besar
3	0.841468253968254	0.000002730839643	lebih kecil
4	0.841471009700176	0.000000024892280	lebih besar
5	0.841470984648068	0.000000000159828	lebih kecil

Pengamatan:

- Galat menurun sangat cepat.** Galat turun dari orde 10^{-3} ($n = 1$) ke 10^{-10} ($n = 5$). Ini menunjukkan deret **konvergen** ke $\sin(x)$. Pada $n = 5$, hasilnya benar sampai 9 angka desimal (0,841470984...).
- Pendekatan berganti-ganti di bawah dan di atas nilai eksak.** Ini terjadi karena tanda suku deret bergantian. Selisih tanda ini tidak terlihat di kolom galat, karena $\text{abs}()$ selalu menghasilkan nilai positif. Deret dengan tanda bergantian (alternating series) inilah yang membedakannya dari deret e^x pada program sebelumnya, di mana pendekatan selalu berada di bawah nilai eksak.
- Galat sedikit lebih kecil dari suku pertama yang dibuang.** Untuk deret bergantian, galat pemotongan dibatasi oleh suku pertama yang dibuang:

n	Suku pertama yang dibuang	Galat sebenarnya
1	$1/5! = 0.008333333$	0.008137651
2	$1/7! = 0.000198413$	0.000195682
3	$1/9! = 0.0000027557$	0.0000027308
4	$1/11! = 0.000000025052$	0.000000024892

5	$1/13! = 0.00000000016059$	0.00000000015983
---	----------------------------	------------------

4. Batas ini juga bisa dipakai untuk menentukan n minimum sebelum program dijalankan: $|R_n| \leq x^{(2n+3)}/(2n+3)!$.
5. **Setiap penambahan n memperkecil galat jauh lebih besar dibanding e^x .** Rasio galat antar n berturut-turut adalah 41,6; 71,7; 109,7; dan 155,7. Angka ini sangat dekat dengan $(2n+4)(2n+5)$, yaitu 42, 72, 110, dan 156. Alasannya, satu penambahan n menambah **dua** orde pangkat ($x^{(2i+1)}$) sekaligus, dan faktorialnya naik dua tingkat.
6. **Verifikasi manual.** Untuk $n = 1$, $p = 1 - 1/6 = 0,833333...$ sesuai output. Untuk $n = 2$, $p = 0,833333 + 1/120 = 0,841666...$, juga sesuai output.
7. **Catatan untuk x yang besar.** Deret Maclaurin paling akurat untuk x kecil (dekat 0). Untuk x besar (misalnya $x = 10$), suku-suku awal sangat besar dan bergantian tanda, sehingga n harus dinaikkan jauh lebih tinggi dan muncul risiko **galat pembulatan akibat pengurangan bilangan hampir sama**. Praktisnya, x direduksi dulu ke rentang $[-\pi, \pi]$ dengan sifat periodik $\sin(x)$.

BAB II

ANALISIS NUMERIK

Galat Hampiran $e^{0,3}$ Menggunakan Deret Taylor

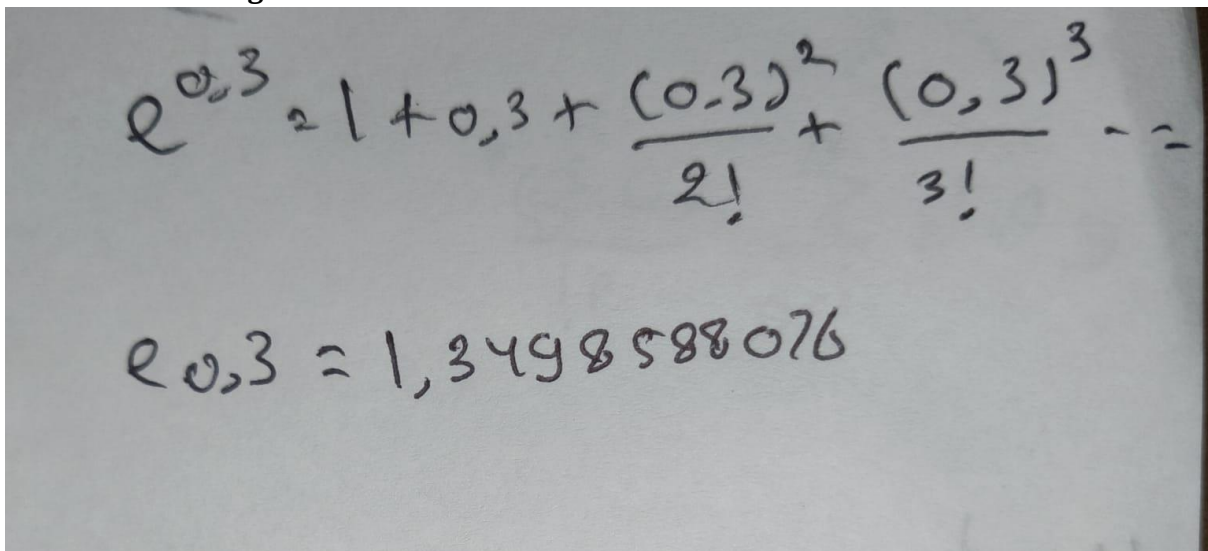
1. Soal

Tentukan nilai galat (error) pada perhitungan $e^{0,3}$ menggunakan deret Taylor:

$$e^x = \sum (\text{dari } i = 0 \text{ sampai } n) x^i / i!$$

dengan $n = \{0, 1, 2, 3, 4\}$, kemudian bandingkan dan analisis hasil yang diperoleh.

2. Dasar Perhitungan



The image shows a handwritten calculation of $e^{0,3}$ using the Taylor series expansion. The formula is written as:

$$e^{0,3} = 1 + 0,3 + \frac{(0,3)^2}{2!} + \frac{(0,3)^3}{3!} + \dots$$

Below this, the result is given as:

$$e^{0,3} = 1,3498588076$$

Nilai $x = 0,3$. Nilai sebenarnya (pembanding) diambil dari kalkulator ilmiah:

$$e^{0,3} = 1,3498588076$$

Galat dihitung dengan dua ukuran:

Galat absolut: $E = | \text{nilai sebenarnya} - \text{nilai hampiran} |$

Galat relatif: $E_r = (E / \text{nilai sebenarnya}) \times 100\%$

3. Perhitungan

Suku-suku deret untuk $x = 0,3$:

$$i = 0 : 0,3^0 / 0! = 1 / 1 = 1$$

$$i = 1 : 0,3^1 / 1! = 0,3 / 1 = 0,3$$

$$i = 2 : 0,3^2 / 2! = 0,09 / 2 = 0,045$$

$$i = 3 : 0,3^3 / 3! = 0,027 / 6 = 0,0045$$

$$i = 4 : 0,3^4 / 4! = 0,0081 / 24 = 0,0003375$$

a. n = 0

Untuk $n = 0$

$$e^{0.3} \approx \sum_{i=0}^0 \frac{(0.3)^i}{i!}$$

$$e^{0.3} \approx \frac{(0.3)^0}{0!}$$

$$(0.3)^0 = 1$$

$$0! = 1$$

$$\frac{1}{1} = 1$$

$$e^{0.3} \approx 1.3498588076$$

$$E = |1.3498588076 - 1|$$

$$E = 0.3498588076$$

$$e^{0.3} \approx 1$$

$$E = |1.3498588076 - 1| = 0.3498588076$$

$$E_r = (0.3498588076 / 1.3498588076) \times 100\% = 25.918\%$$

b. n = 1

Untuk $n = 1$

$$e^{0.3} \approx \sum_{i=0}^1 \frac{(0.3)^i}{i!}$$

$$= \frac{(0.3)^0}{0!} + \frac{(0.3)^1}{1!}$$

$$= \frac{1}{1} + \frac{0.3}{1}$$

$$= 1 + 0.3 = 1.3$$

$$e^{0.3} \approx 1.3$$

$$E = |1.3498588076 - 1.3|$$

$$E = 0.0498588076$$

$$e^{0.3} \approx 1 + 0.3 = 1.3$$

$$E = |1.3498588076 - 1.3| = 0.0498588076$$

$$E_r = (0.0498588076 / 1.3498588076) \times 100\% = 3.694\%$$

c. n = 2

Untuk $n = 2$
$$e^{0.3} \approx \sum_{i=0}^2 \frac{(0.3)^i}{i!}$$
$$= \frac{(0.3)^0}{0!} + \frac{(0.3)^1}{1!} + \frac{(0.3)^2}{2!}$$
$$0.3^2 = 0.09$$
$$2! = 2 \times 1 = 2$$
$$\frac{0.09}{2} = 0.045$$
$$= 1 + 0.3 + 0.045$$
$$e^{0.3} = 1.345$$
$$E = 1.3498588076 - 1.345$$
$$E = 0.0048588076$$

$$e^{0.3} \approx 1 + 0.3 + 0.045 = 1.345$$

$$E = |1.3498588076 - 1.345| = 0.0048588076$$

$$E_r = (0.0048588076 / 1.3498588076) \times 100\% = 0.360\%$$

d. n = 3

Untuk $n = 3$
$$e^{0.3} \approx \sum_{i=0}^3 \frac{(0.3)^i}{i!}$$
$$0.3^3 = 0.027$$
$$3! = 6$$
$$\frac{0.027}{6} = 0.0045$$
$$= 1 + 0.3 + 0.045 + 0.0045$$
$$e^{0.3} = 1.3495$$
$$E = 1.3498588076 - 1.3495$$
$$E = 0.0003588076$$

$$e^{0.3} \approx 1 + 0.3 + 0.045 + 0.0045 = 1.3495$$

$$E = |1.3498588076 - 1.3495| = 0.0003588076$$

$$E_r = (0.0003588076 / 1.3498588076) \times 100\% = 0.02658\%$$

e. n = 4

$$e^{0.3} \approx \sum_{i=0}^4 \frac{(0.3)^i}{i!}$$
$$0.3^4 = 0.3 \times 0.3 \times 0.3 \times 0.3$$
$$= 0.0081$$
$$4! = 24$$
$$\frac{0.0081}{24} = 0.0003375$$
$$= 1 + 0.3 + 0.045 + 0.0045 + 0.0003375$$
$$e^{0.3} = 1.3498375$$
$$E = 1.3498588076 - 1.3498375$$
$$E = 0.0000213076$$

$$e^{0,3} \approx 1 + 0,3 + 0,045 + 0,0045 + 0,0003375 = 1,3498375$$

$$E = |1,3498588076 - 1,3498375| = 0,0000213076$$

$$E_r = (0,0000213076 / 1,3498588076) \times 100\% = 0,001578\%$$

4. Tabel Perbandingan

n	Hampiran Taylor	Galat absolut (E)	Galat relatif (E _r)
0	1,0000000	0,3498588	25,918%
1	1,3000000	0,0498588	3,694%
2	1,3450000	0,0048588	0,360%
3	1,3495000	0,0003588	0,02658%
4	1,3498375	0,0000213	0,001578%

Galat Penjumlahan $1/1 + 1/2 + 1/3 + \dots + 1/20$

1. Soal

Tentukan nilai galat (error) pada perhitungan:

$$S = 1/1 + 1/2 + 1/3 + \dots + 1/20$$

Perhitungan dilakukan dengan tiga cara, kemudian hasilnya dibandingkan:

- Perhitungan secara eksak
- Masing-masing pembagian dibulatkan
- Tanpa looping (menggunakan fungsi sum)

2. Dasar Perhitungan

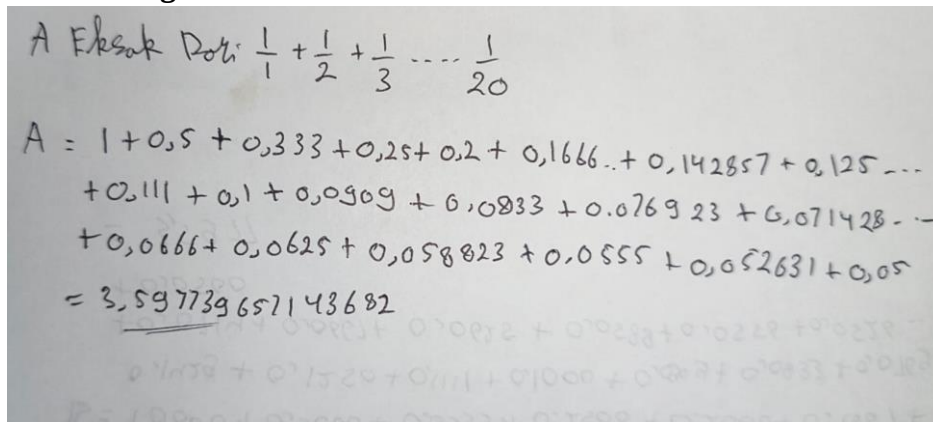
Hasil perhitungan eksak dijadikan nilai acuan. Galat dihitung dengan:

Galat absolut: $E = | \text{nilai eksak} - \text{nilai hampiran} |$

Galat relatif: $E_r = (E / \text{nilai eksak}) \times 100\%$

3. Perhitungan

a. Perhitungan secara eksak



A Eksak Dari $\frac{1}{1} + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{20}$

$$\begin{aligned} A &= 1 + 0,5 + 0,333 + 0,25 + 0,2 + 0,1666 + 0,142857 + 0,125 + \dots \\ &\quad + 0,111 + 0,1 + 0,0909 + 0,0833 + 0,076923 + 0,071428 + \dots \\ &\quad + 0,0666 + 0,0625 + 0,058823 + 0,0555 + 0,052631 + 0,05 \\ &= \underline{3,597739657143682} \end{aligned}$$

Metode eksak dimulai dengan mengubah bilangan pecahan ke bilangan bulat

$$1/1=1$$

$1/2=0,5$ dst lalu di tambahkan dari $1/1$ sampai $1/20$ dan didapatkan hasil= $3,597739657143682$

b. Masing-masing pembagian dibulatkan

Nilai tiap suku $1/k$:

B. Dibulatkan

$$\begin{aligned} B &= 1,0000 + 0,5000 + 0,3333 + 0,2500 + 0,2000 + 0,1667 + \\ &\quad 0,1429 + 0,1250 + 0,1111 + 0,1000 + 0,0909 + 0,0833 + 0,0769 + \\ &\quad + 0,0714 + 0,0667 + 0,0625 + 0,0589 + 0,0556 + 0,0526 + \\ &\quad + 0,0500 \\ &= 3,5977 \end{aligned}$$

Seperti yang pertama namun dibulatkan menjadi 4 desimal

$$1/1=1=1,0000$$

$$1/2=0,5=0,5000$$

selengkapnya bisa ddi lihat di table di bawah ini

k	1/k (eksak, 10 desimal)	Dibulatkan 4 desimal	Dibulatkan 6 desimal
1	1,0000000000	1,0000	1,000000
2	0,5000000000	0,5000	0,500000
3	0,3333333333	0,3333	0,333333
4	0,2500000000	0,2500	0,250000
5	0,2000000000	0,2000	0,200000
6	0,1666666667	0,1667	0,166667
7	0,1428571429	0,1429	0,142857
8	0,1250000000	0,1250	0,125000
9	0,1111111111	0,1111	0,111111
10	0,1000000000	0,1000	0,100000
11	0,0909090909	0,0909	0,090909
12	0,0833333333	0,0833	0,083333
13	0,0769230769	0,0769	0,076923
14	0,0714285714	0,0714	0,071429
15	0,0666666667	0,0667	0,066667

16	0,0625000000	0,0625	0,062500
17	0,0588235294	0,0588	0,058824
18	0,0555555556	0,0556	0,055556
19	0,0526315789	0,0526	0,052632
20	0,0500000000	0,0500	0,050000

Penjumlahan dengan pembulatan 4 angka di belakang koma:

$$k = 1-5 : 1,0000 + 0,5000 + 0,3333 + 0,2500 + 0,2000 = 2,2833$$

$$k = 6-10 : 0,1667 + 0,1429 + 0,1250 + 0,1111 + 0,1000 = 0,6457$$

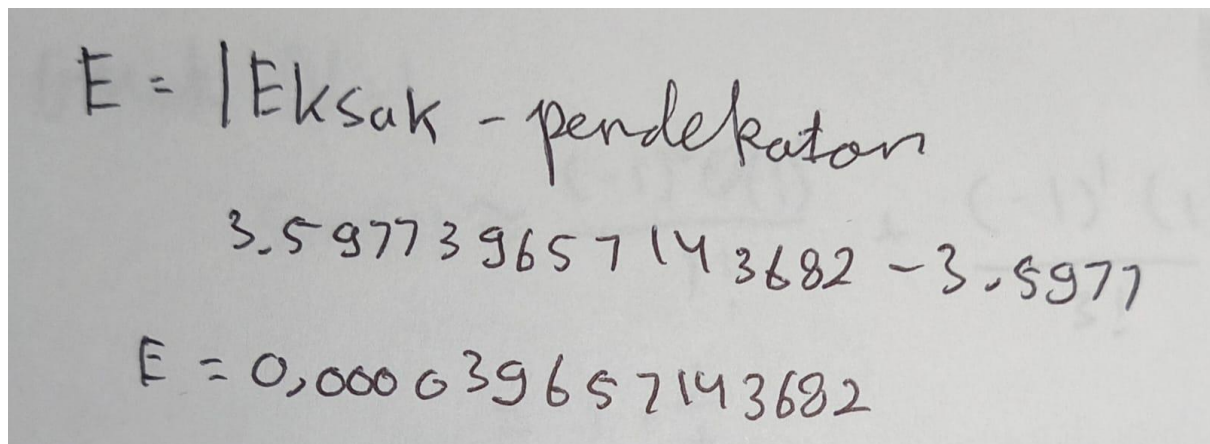
$$k = 11-15 : 0,0909 + 0,0833 + 0,0769 + 0,0714 + 0,0667 = 0,3892$$

$$k = 16-20 : 0,0625 + 0,0588 + 0,0556 + 0,0526 + 0,0500 = 0,2795$$

$$S \approx 2,2833 + 0,6457 + 0,3892 + 0,2795 = 3,5977$$

$$E = |3,5977396571 - 3,5977| = 0,0000396571$$

C. hitung galat (EKSAK – PEMBULATAN)



Handwritten calculation showing the error E as the absolute difference between the exact value and the approximation:

$$E = |Eksak - pendekatan|$$

$$3.597739657143682 - 3.5977$$

$$E = 0,000039657143682$$

$$E_r = 3.597739657143682 - 3.5977$$

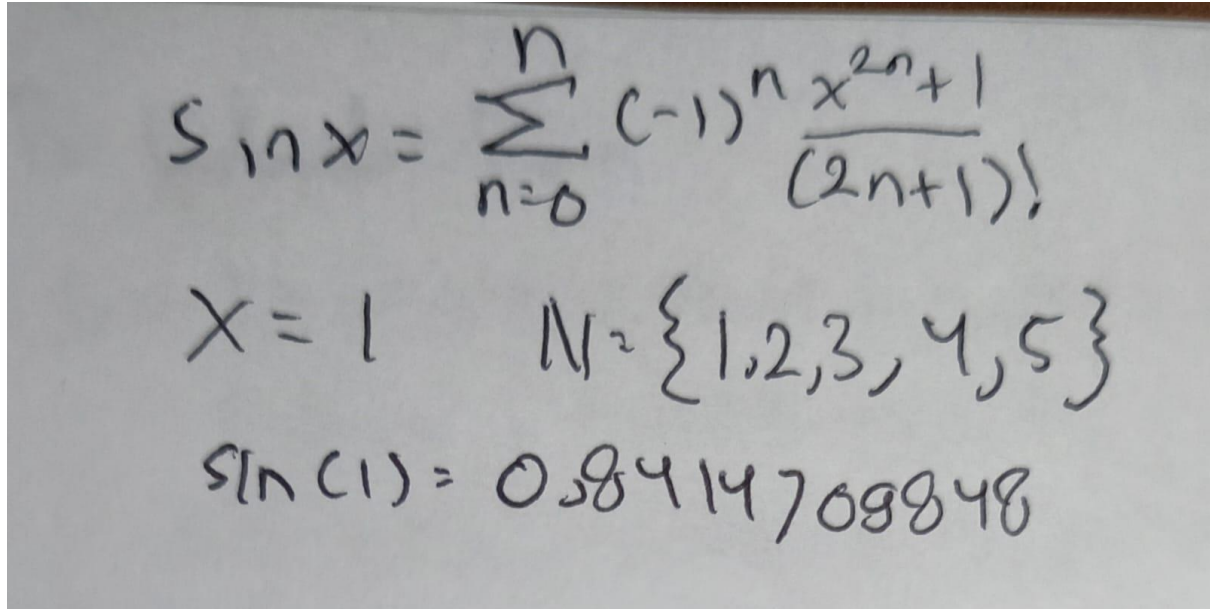
1. Soal

Hitung hampiran nilai $\sin(x)$ untuk $x = 1$ (radian) dengan deret Taylor:

$$\sin x = \sum_{n=0}^N (-1)^n x^{2n+1} / (2n+1)!$$

untuk $N = \{1, 2, 3, 4, 5\}$, kemudian tentukan galatnya dan analisis hasilnya.

2. Dasar Perhitungan


$$\sin x = \sum_{n=0}^N (-1)^n \frac{x^{2n+1}}{(2n+1)!}$$
$$x = 1 \quad N = \{1, 2, 3, 4, 5\}$$
$$\sin(1) = 0.8414709848$$

Nilai eksak (pembanding) diambil dari kalkulator ilmiah dengan mode radian:

$$\sin(1) = 0,8414709848$$

Galat absolut: $E = | \text{nilai eksak} - \text{nilai hampiran} |$

Galat relatif: $E_r = (E / \text{nilai eksak}) \times 100\%$

Suku-suku deret untuk $x = 1$:

$$n = 0 : (-1)^0 \cdot 1^1 / 1! = 1$$

$$n = 1 : (-1)^1 \cdot 1^3 / 3! = -1/6 = -0,1666666667$$

$$n = 2 : (-1)^2 \cdot 1^5 / 5! = +1/120 = +0,0083333333$$

$$n = 3 : (-1)^3 \cdot 1^7 / 7! = -1/5040 = -0,0001984127$$

$$n = 4 : (-1)^4 \cdot 1^9 / 9! = +1/362880 = +0,0000027557$$

$$n = 5 : (-1)^5 \cdot 1^{11} / 11! = -1/39916800 = -0,0000000251$$

3. Perhitungan

a. $N = 1$

Untuk $N=1$

$$\sin(1) \approx \frac{(-1)^0 (1)^1}{1!} + \frac{(-1)^1 (1)^3}{3!}$$
$$= 1 - \frac{1}{6}$$
$$= 0,8333333333$$
$$E = 0,8414709848 - 0,8333333333$$
$$E = 0,0081376515$$

$$\sin(1) \approx 1 - 1/6 = 5/6 = 0,8333333333$$

$$E = |0,8414709848 - 0,8333333333| = 0,0081376515$$

$$E_r = (0,0081376515 / 0,8414709848) \times 100\% = 0,9671\%$$

b. $N = 2$

Untuk $N=2$

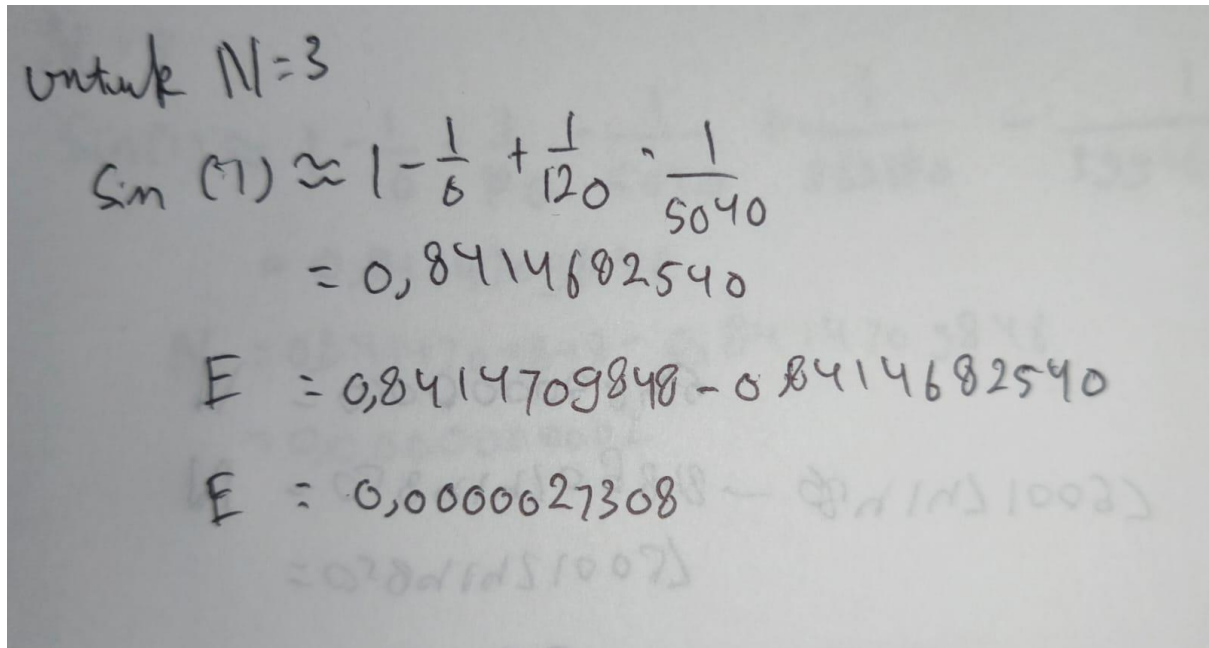
$$\sin(1) \approx 1 - \frac{1}{3!} + \frac{1}{5!}$$
$$= 1 - \frac{1}{6} + \frac{1}{120}$$
$$= 0,8416666667$$
$$E = 0,8414709848 - 0,8416666667$$
$$E = 0,0001956819$$

$$\sin(1) \approx 5/6 + 1/120 = 100/120 + 1/120 = 101/120 = 0,8416666667$$

$$E = |0,8414709848 - 0,8416666667| = 0,0001956819$$

$$E_r = (0,0001956819 / 0,8414709848) \times 100\% = 0,02325\%$$

c. $N = 3$



Handwritten calculations for $N=3$:

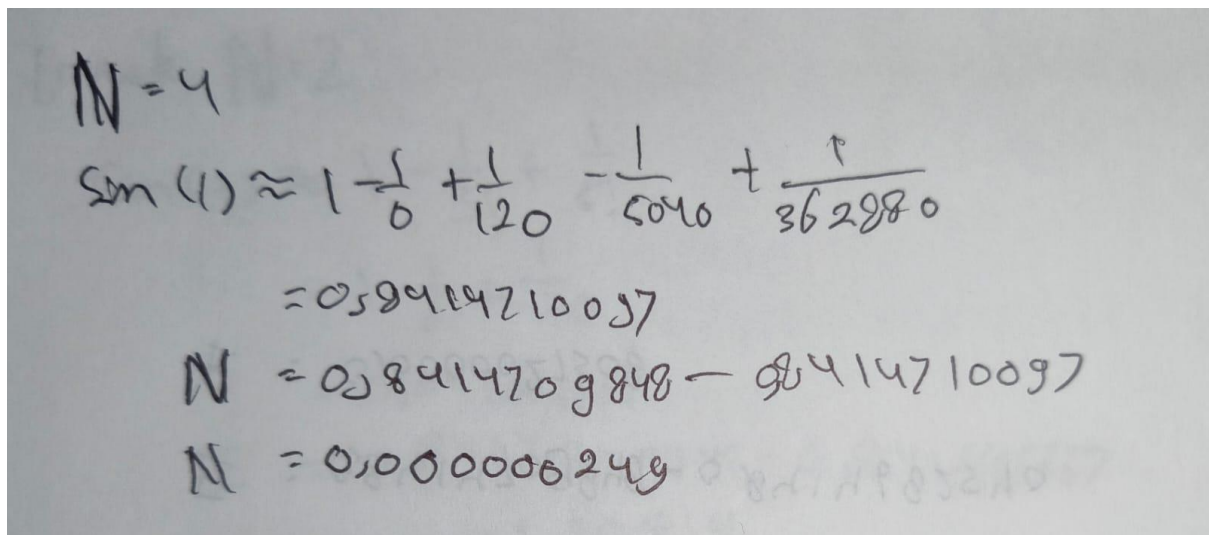
$$\sin(1) \approx 1 - \frac{1}{6} + \frac{1}{120} - \frac{1}{5040}$$
$$= 0,8414682540$$
$$E = 0,8414709848 - 0,8414682540$$
$$E = 0,0000027308$$

$$\sin(1) \approx 101/120 - 1/5040 = 4242/5040 - 1/5040 = 4241/5040 = 0,8414682540$$

$$E = |0,8414709848 - 0,8414682540| = 0,0000027308$$

$$E_r = (0,0000027308 / 0,8414709848) \times 100\% = 0,0003245\%$$

d. $N = 4$



Handwritten calculations for $N=4$:

$$\sin(1) \approx 1 - \frac{1}{6} + \frac{1}{120} - \frac{1}{5040} + \frac{1}{362880}$$
$$= 0,8414710097$$
$$N = 0,8414709848 - 0,8414710097$$
$$N = 0,000000249$$

$$\sin(1) \approx 0,8414682540 + 0,0000027557 = 0,8414710097$$

$$E = |0,8414709848 - 0,8414710097| = 0,000000249$$

$$E_r = (0,000000249 / 0,8414709848) \times 100\% = 0,000002958\%$$

e. $N = 5$

$$\begin{aligned} N=5 \\ \sin(1) &\approx 1 - \frac{1}{6} + \frac{1}{120} - \frac{1}{5040} + \frac{1}{362880} - \frac{1}{39916800} \\ &= 0,8414709846 \\ N &= 0,8414709848 - 0,8414709846 \\ &= 0,0000000002 \end{aligned}$$

$$\sin(1) \approx 0,8414710097 - 0,00000000251 = 0,8414709846$$

$$E = |0,8414709848 - 0,8414709846| \approx 0,0000000002 \quad (1,6 \times 10^{-10})$$

$$E_r \approx 1,9 \times 10^{-8} \%$$

4. Tabel Perbandingan

N	Hampiran Taylor	Galat absolut (E)	Galat relatif (E_r)
1	0,8333333333	0,0081376515	0,9671%
2	0,8416666667	0,0001956819	0,02325%
3	0,8414682540	0,0000027308	0,0003245%
4	0,8414710097	0,0000000249	0,000002958%
5	0,8414709846	0,0000000002	$1,9 \times 10^{-8} \%$

5. Analisis

a. Semakin besar N , semakin kecil galatnya. Penambahan jumlah suku pada deret Taylor membuat hasil hampiran semakin mendekati nilai eksak $\sin(1) = 0,8414709848$. Galat absolut turun dari 0,0081376515 ($N = 1$) menjadi sekitar 0,0000000002 ($N = 5$).

b. Penurunan galat semakin cepat pada setiap penambahan suku, yaitu sekitar 41,6 kali ($N = 1$ ke 2), 71,7 kali ($N = 2$ ke 3), 109,7 kali ($N = 3$ ke 4), dan 155,6 kali ($N = 4$ ke 5). Hal ini terjadi karena penyebut $(2n+1)!$ bertambah sangat cepat.

c. Hasil hampiran berada bergantian di bawah dan di atas nilai eksak ($N = 1$ di bawah, $N = 2$ di atas, $N = 3$ di bawah, dan seterusnya) karena tanda suku deret bergantian. Pada deret

bergantian seperti ini, galat tidak melebihi nilai suku pertama yang tidak dihitung, seperti terlihat pada tabel berikut.

N	Suku ke-(N+1) yang dipotong	Galat absolut (E)
1	$1/120 = 0,0083333333$	0,0081376515
2	$1/5040 = 0,0001984127$	0,0001956819
3	$1/362880 = 0,0000027557$	0,0000027308
4	$1/39916800 = 0,0000000251$	0,0000000249
5	$1/13! = 0,00000000016$	0,00000000016

d. Untuk $x = 1$, deret Taylor konvergen cepat. Pada $N = 5$, hampiran 0,8414709846 sudah sama dengan nilai eksak sampai sembilan angka di belakang koma (0,841470984).

6. Kesimpulan

Galat hampiran $\sin(1)$ dengan deret Taylor menurun tajam seiring bertambahnya N , dari 0,0081376515 pada $N = 1$ menjadi sekitar 0,0000000002 pada $N = 5$. Semakin banyak suku yang digunakan, semakin akurat hasilnya.

BAB III

ANALISIS PRAKTIKUM

1. Perbandingan Hasil

Hasil analisis source code (Octave) dibandingkan dengan hasil analisis numerik (perhitungan manual) untuk tiga permasalahan. Perhitungan manual memakai 10 angka di belakang koma, sedangkan Octave menampilkan 15 angka di belakang koma (presisi ganda).

a. Deret Taylor $e^{0,3}$

n	Galat manual	Galat Octave	Selisih	Posisi terhadap eksak
0	0,3498588076	0,349858807576003	$2,4 \times 10^{-11}$	lebih kecil
1	0,0498588076	0,049858807576003	$2,4 \times 10^{-11}$	lebih kecil
2	0,0048588076	0,004858807576003	$2,4 \times 10^{-11}$	lebih kecil
3	0,0003588076	0,000358807576003	$2,4 \times 10^{-11}$	lebih kecil
4	0,0000213076	0,000021307576003	$2,4 \times 10^{-11}$	lebih kecil

b. Penjumlahan $1/1 + 1/2 + \dots + 1/20$

Metode	Hasil manual	Hasil Octave	Galat
a. Eksak (loop)	3,5977396571	3,5977396571	0
b. Dibulatkan 4 desimal	3,5977	3,5977000000	0,0000396571
c. Tanpa loop (sum)	3,5977396571	3,5977396571	≈ 0

c. Deret Taylor $\sin(1)$

N	Galat manual	Galat Octave	Selisih	Posisi terhadap eksak
1	0,0081376515	0,008137651474563	$2,5 \times 10^{-11}$	lebih kecil
2	0,0001956819	0,000195681858770	$4,1 \times 10^{-11}$	lebih besar
3	0,0000027308	0,000002730839643	$4,0 \times 10^{-11}$	lebih kecil
4	0,0000000249	0,000000024892280	$7,7 \times 10^{-12}$	lebih besar

5	0,00000000002	0,0000000000159828	$4,0 \times 10^{-11}$	lebih kecil
---	---------------	--------------------	-----------------------	-------------

2. Analisis Perbandingan

a. Seluruh hasil manual sesuai dengan hasil source code. Selisihnya kurang dari 5×10^{-11} dan hanya muncul pada angka desimal ke-11 atau lebih. Selisih ini berasal dari pembulatan pada perhitungan manual (10 desimal), sedangkan Octave memakai presisi ganda dengan ketelitian sekitar 15 sampai 16 angka signifikan.

b. Deret Taylor $e^{0,3}$. Source code dan hitungan manual sama-sama menunjukkan galat pemotongan yang menurun dari orde 10^{-1} ($n = 0$) ke orde 10^{-5} ($n = 4$), dengan penurunan sekitar 7 sampai 17 kali pada setiap penambahan satu suku. Karena $x = 0,3 > 0$, semua suku deret positif sehingga hampiran selalu lebih kecil dari nilai eksak. Galat kira-kira sama dengan suku pertama yang dibuang; pada $n = 4$ suku berikutnya adalah $0,3^5/5! = 0,00002025$, sedangkan galatnya 0,0000213.

c. Deret Taylor $\sin(1)$. Galat turun dari orde 10^{-3} ($N = 1$) ke orde 10^{-10} ($N = 5$), dan pada $N = 5$ hasilnya benar sampai 9 angka desimal (0,841470984). Hampiran berada bergantian di bawah dan di atas nilai eksak karena tanda suku deret bergantian, berbeda dengan $e^{0,3}$. Galat pemotongan tidak melebihi suku pertama yang dibuang (misalnya $N = 5$: $1/13! = 1,6059 \times 10^{-10}$ dan galat $1,5983 \times 10^{-10}$). Rasio galat antar N berturut-turut adalah 41,6; 71,7; 109,7; dan 155,7, sangat dekat dengan $(2N+4)(2N+5) = 42, 72, 110$, dan 156.

d. Penjumlahan $1/1 + \dots + 1/20$. Hasil dengan loop dan dengan fungsi sum sama sampai 10 angka di belakang koma (3,5977396571), dan cocok dengan nilai rasional eksak $55835135/15519504 \approx 3,59773965714$. Pembulatan setiap suku ke 4 desimal menghasilkan 3,5977 dengan galat absolut 0,0000396571 (galat relatif sekitar 0,0011%), sama dengan hitungan manual. Galat tiap suku paling besar 5×10^{-5} , sehingga batas terburuk untuk 20 suku adalah 10^{-3} . Galat sebenarnya jauh lebih kecil karena sebagian suku dibulatkan ke atas dan sebagian ke bawah sehingga saling meniadakan. Angka nol pada 3,5977000000 bukan tanda ketelitian tinggi, karena hanya 4 desimal yang bermakna.

e. Perhitungan manual berguna untuk memahami langkah dan sumber galat, tetapi terbatas pada jumlah angka desimal tertentu dan rawan salah hitung. Source code menghasilkan nilai yang lebih teliti dan dapat dijalankan untuk nilai n yang berbeda dengan cepat. Vektorisasi (fungsi sum) membuat kode lebih ringkas dan lebih cepat untuk n besar tanpa mengubah ketelitian secara berarti.

3. Kesimpulan

1. Hasil source code Octave sesuai dengan hasil analisis numerik manual pada ketiga permasalahan (deret Taylor $e^{0,3}$, penjumlahan $1/1 + \dots + 1/20$, dan deret Taylor $\sin(1)$). Selisihnya kurang dari 5×10^{-11} dan hanya berasal dari pembulatan pada perhitungan manual.

2. Pada deret Taylor, semakin banyak suku yang digunakan, semakin kecil galatnya: 0,000021307576003 untuk $e^{0,3}$ pada $n = 4$, dan 0,000000000159828 untuk $\sin(1)$ pada $N = 5$. Hampiran $e^{0,3}$ selalu di bawah nilai eksak, sedangkan hampiran $\sin(1)$ berganti-ganti di bawah dan di atas nilai eksak.
3. Galat pemotongan deret Taylor dapat diperkirakan dari suku pertama yang dibuang, baik pada hitungan manual maupun pada hasil source code.
4. Pembulatan pada setiap suku menyebabkan galat yang lebih besar (0,0000396571 untuk 4 desimal) dibandingkan perhitungan tanpa pembulatan, baik dengan loop maupun dengan fungsi sum, yang hasilnya sama dengan nilai eksak sampai 10 desimal.
5. Source code memberi hasil yang lebih teliti dan lebih cepat, sedangkan perhitungan manual berguna untuk membuktikan langkah perhitungan dan memverifikasi hasil program.